LAPORAN JOBSHEET
“9-Server Side Rendering (SSR)”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

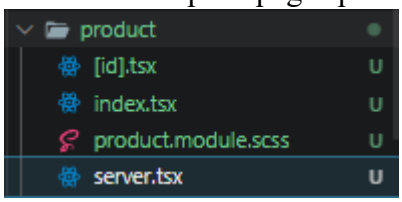
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Bagian 1 – Setup Halaman SSR

1. Buat file baru pada pages/products/server.tsx



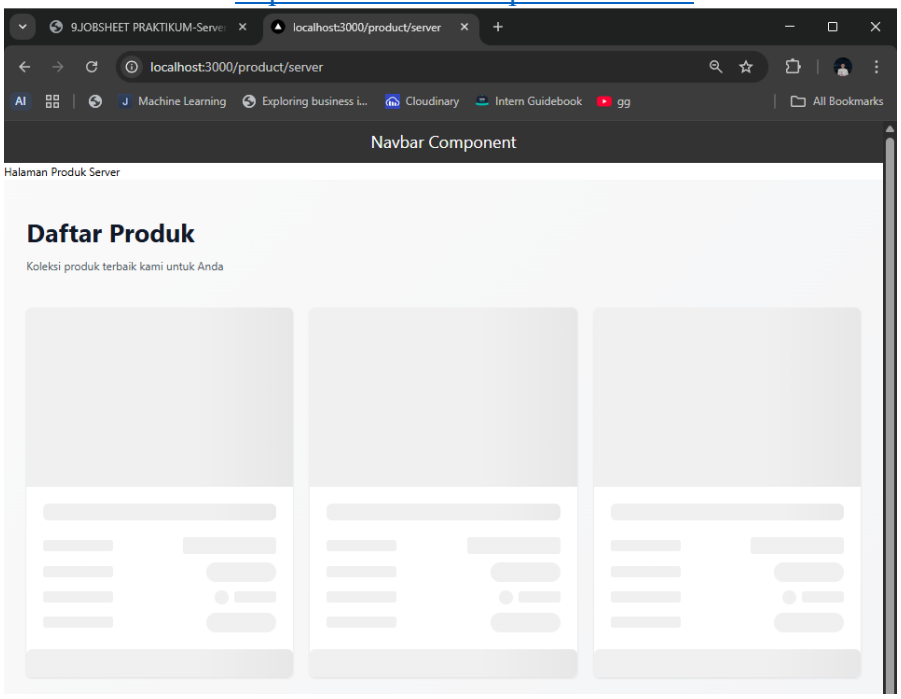
2. Modifikasi server.tsx

```
P2 > Praktikum > next-routing > src > pages > product > server.tsx > ...
import ShowProducts from "../views/product";

const halamanProdukServer = () => {
  return (
    <div>
      <h1>Halaman Produk Server</h1>
      <ShowProducts products={[]} isLoading={true} />
    </div>
  );
};

export default halamanProdukServer;
```

saya menambahkan prop isLoading yg required di index.tsk pada folder views/product, isLoading ini saya gunakan untuk merender tampilan product saya menggunakan fetch pada praktikum sebelumnya. Kemudian

```
{onRefresh && (
  <button
    onClick={onRefresh}
    disabled={isLoading}
    className={styles.refreshButton}
    aria-label="Refresh products"
  >
    {isLoading ? "Loading..." : "Refresh"}
  </button>
```

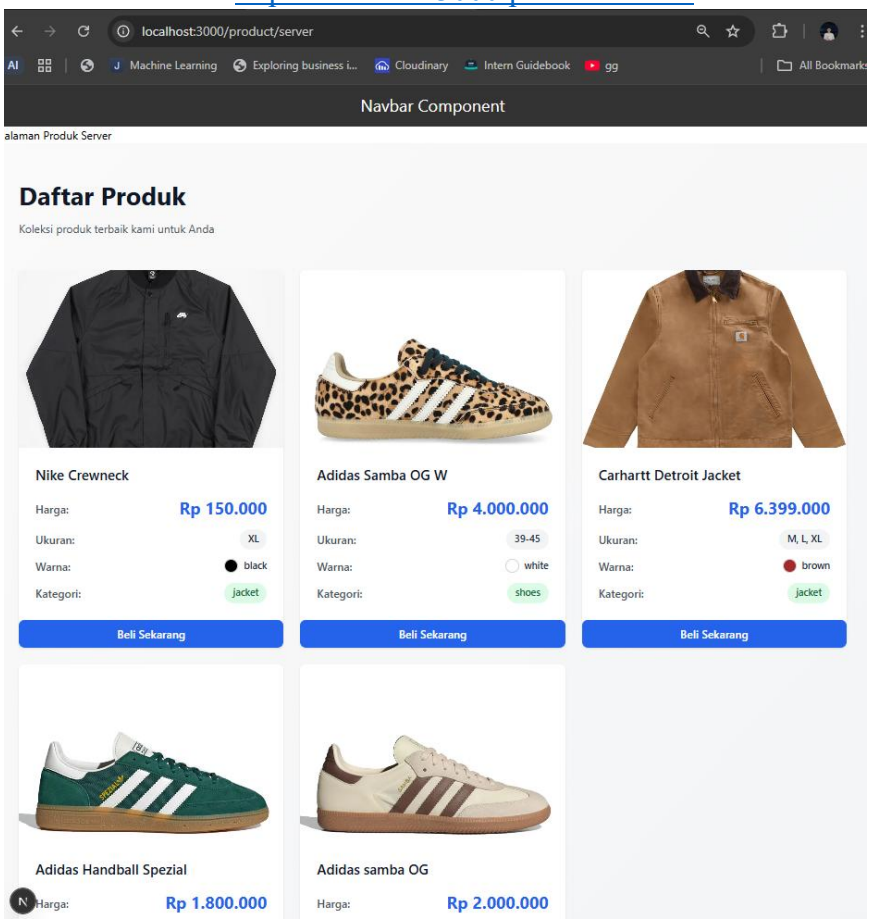
3. Jalankan browser http://localhost:3000/product/server



Bagian 2 – Implementasi getServerSideProps pada server.tsx

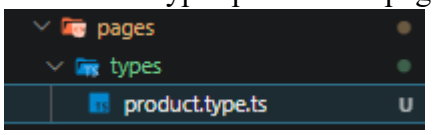
```
server.tsx M ×   index.tsx 2
P2 > Praktikum > next-routing > src > pages > product > server.tsx > ...
3   type ProductType = {
9     category: string;
10    image: string;
11  };
12
13  const halamanProdukServer = (props: { products: ProductType[] }) => {
14    const { products } = props;
15
16    return (
17      <div>
18        <h1>Halaman Produk Server</h1>
19        <ShowProducts products={products} isLoading={false} />
20      </div>
21    );
22  };
23
24  export default halamanProdukServer;
25
26  // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan
27  export const getServerSideProps = async () => {
28    const res = await fetch("http://localhost:3000/api/product");
29    const response = await res.json();
30
31    // console.log("Data produk yang diambil dari API:", response);
32
33    return {
34      props: {
35        products: response.data, // Pastikan untuk memberikan nilai default jika data t
36      },
37    };
38  };
39
```

Jalankan browser http://localhost:3000/product/server



Bagian 3 – Refactor Type (produk type)

1. Buat folder types pada folder pages dan buat file Product.type.ts



2. Modifikasi Product.type.ts

```
P2 > Praktikum > next-routing > src > pages > types > product.type.ts > ...
1  export type ProductType = {
2    id: string;
3    name: string;
4    price: number;
5    size: string;
6    color: string;
7    category: string;
8    image: string;
9  };
```

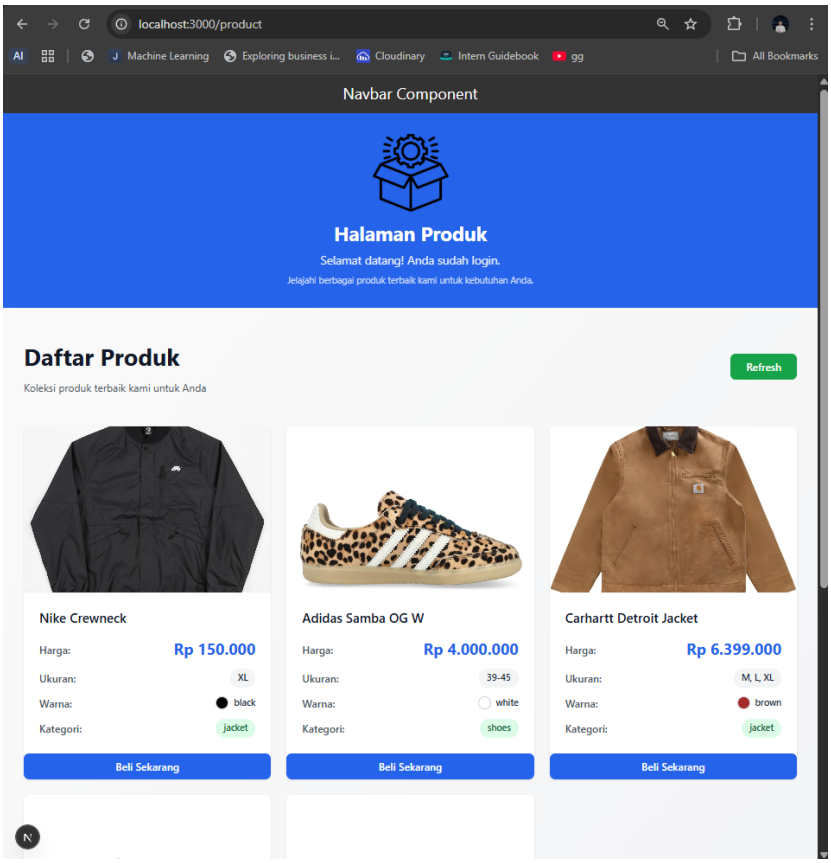
3. Setelah membuat file Product.type.ts maka modifikasi pada file server.tsx menjadi

```
P2 > Praktikum > next-routing > src > pages > product > server.tsx > ...
1  import ShowProducts from "../views/product";
2  import { ProductType } from "../types/product.type";
3
4  const halamanProdukServer = (props: { products: ProductType[] }) => {
5    const { products } = props;
6
7    return (
8      <div>
9        <h1>Halaman Produk Server</h1>
10       <ShowProducts products={products} isLoading={false} />
11     </div>
12   );
13 };
14
15 export default halamanProdukServer;
16
17 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses, dan akan
18 export const getServerSideProps = async () => {
19   const res = await fetch("http://localhost:3000/api/product");
20   const response = await res.json();
21
22   // console.log("Data produk yang diambil dari API:", response);
23
24   return {
25     props: {
26       products: response.data, // Pastikan untuk memberikan nilai default jika data t
27     },
28   };
29 };
```

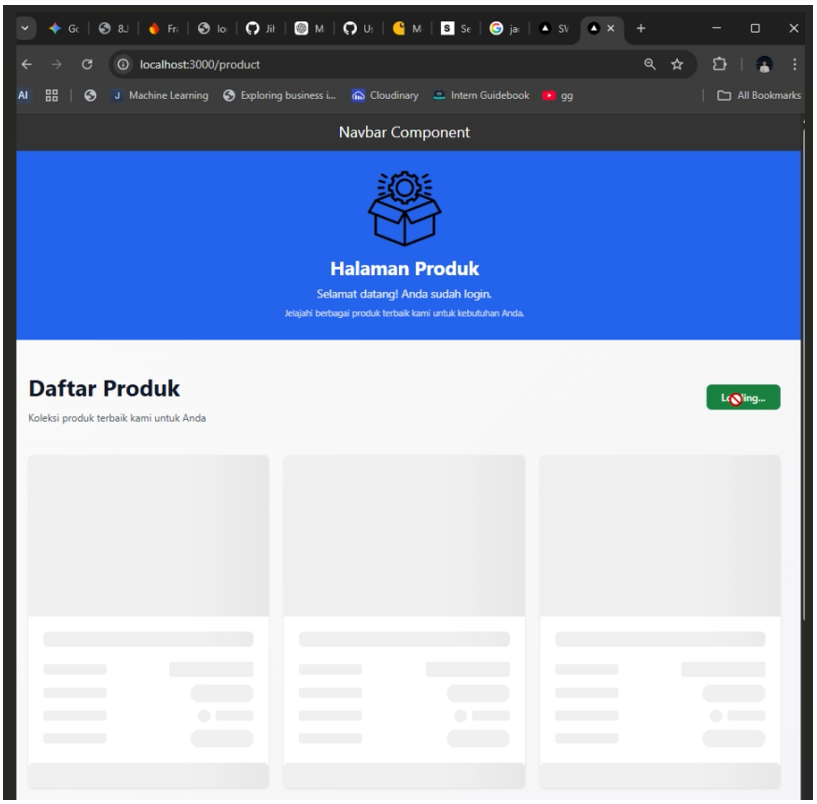
Bagian 4 – Uji Perbedaan SSR vs CSR

1. Uji 1 – Skeleton

- Buka halaman CSR

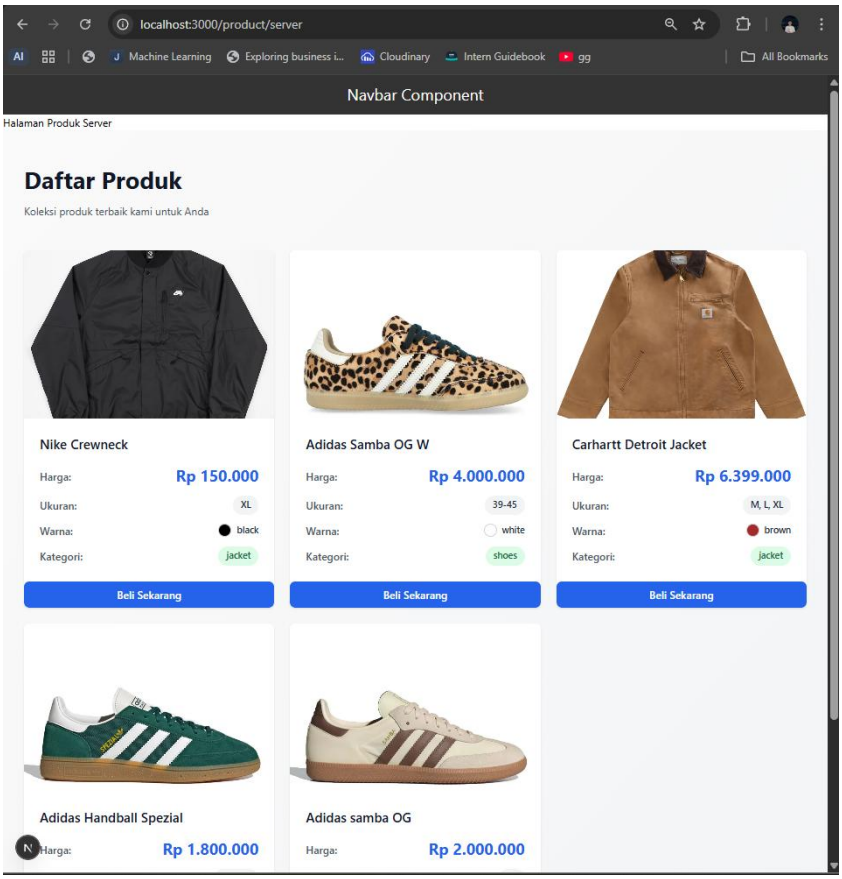


- Refresh
- Skeleton Muncul

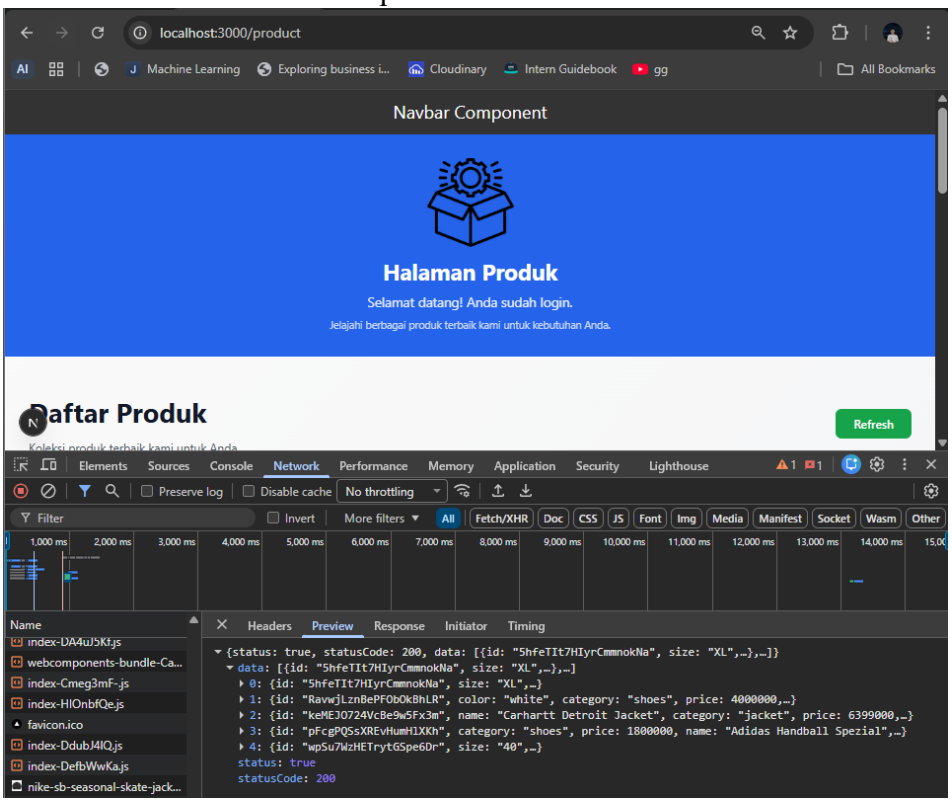


- Buka halaman SSR
- Refresh

- Skeleton tidak muncul



2. Uji 2 – Network Tab
   - Buka DevTools → Network → XHR
   - Refresh halaman CSR → Request API terlihat



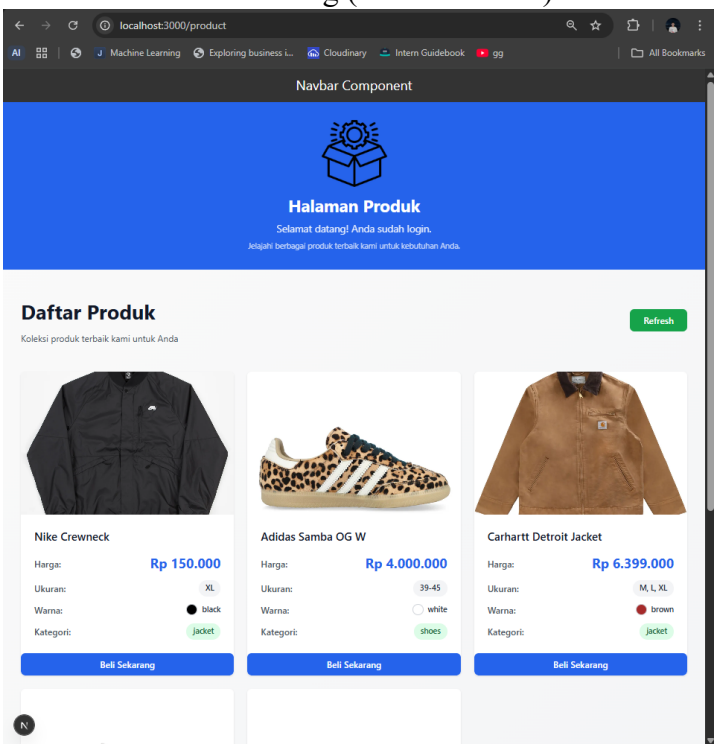
product	304	fetch	VM489:1	0.2 kB	100 ms

- Refresh halaman SSR → Request API tidak terlihat

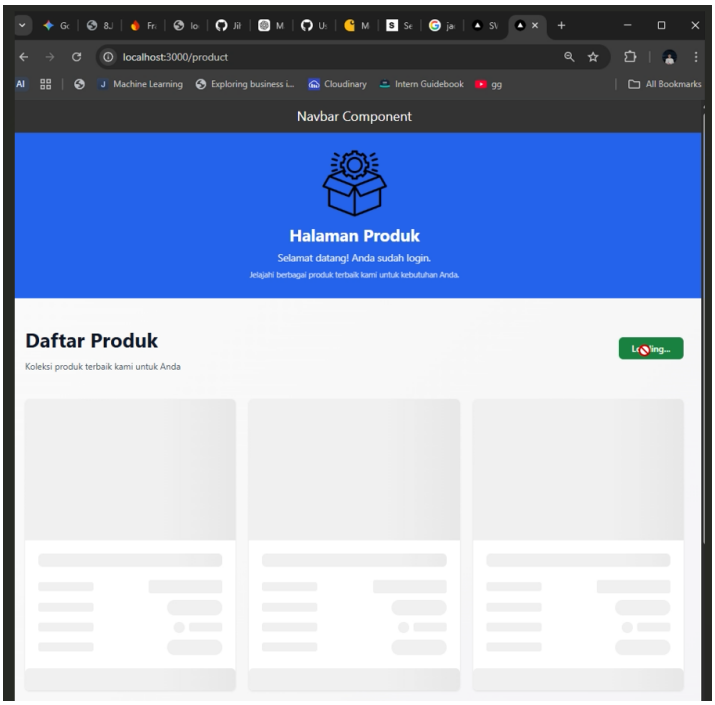
webpack-hmr	101	websocket	websocket.ts:91	0.0 kB	Pending
_devMiddlewareManifest.json	200	fetch	VM532:1	0.2 kB	3 ms
content-all.css	200	fetch	content-all.js:176	523 kB	3 ms
fonts.css	200	fetch	content-all.js:176	6.7 kB	3 ms
127.0.0.1	101	websocket	VM532:1	0.0 kB	Pending

Tidak ada product dengan type fetch

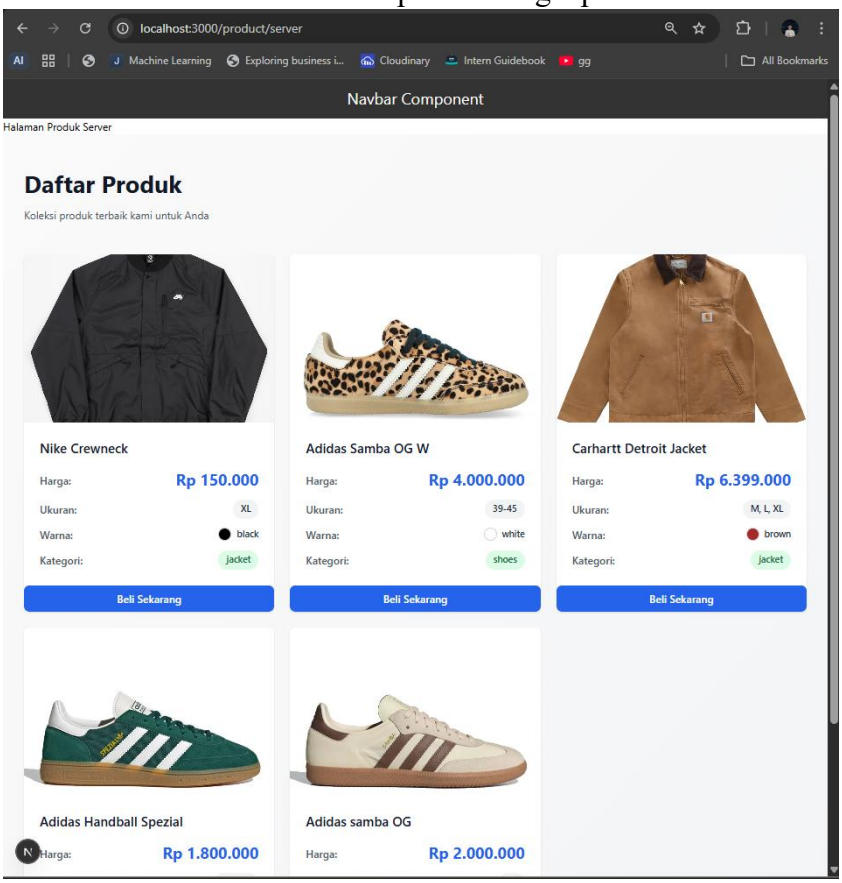
3. Uji 3 – Response HTML
   - CSR: HTML awal kosong (berisi skeleton)



Jika di refresh



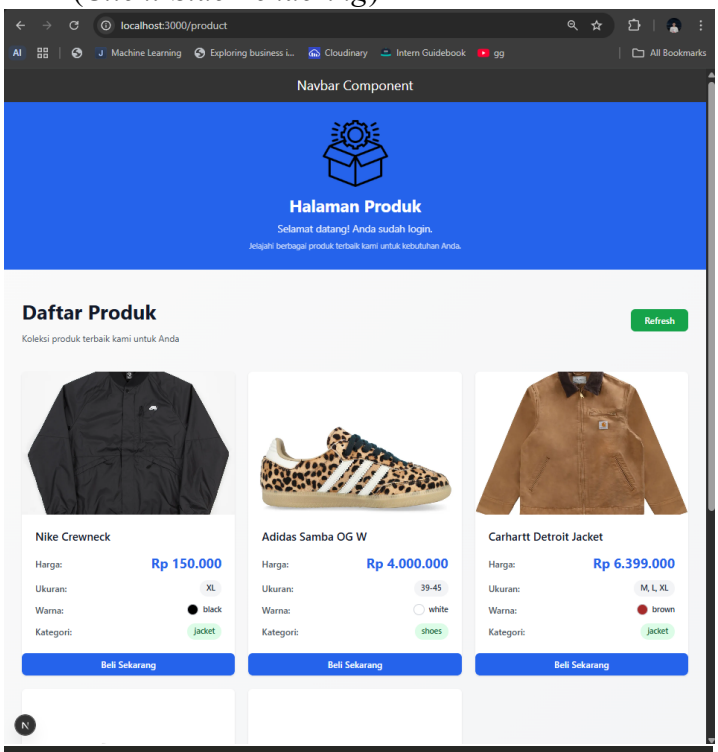
- SSR: HTML sudah berisi data produk lengkap

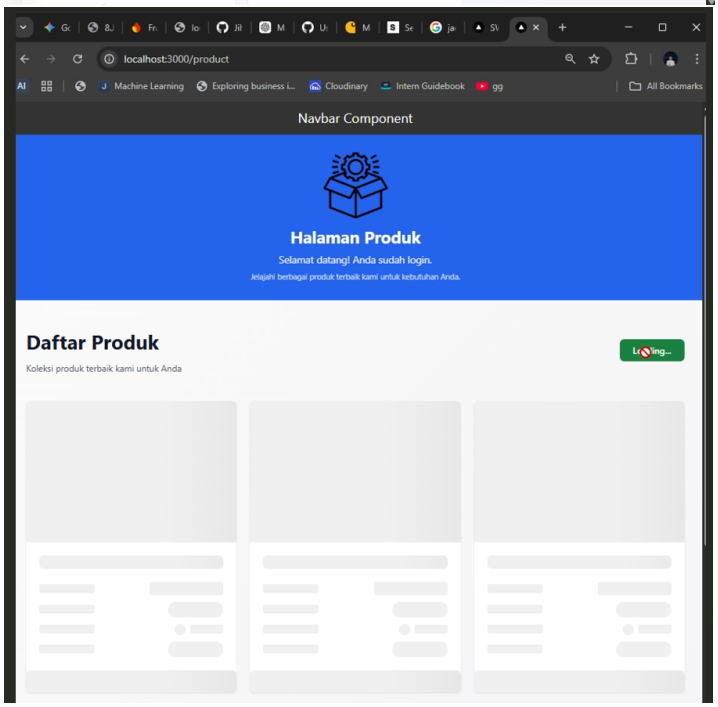


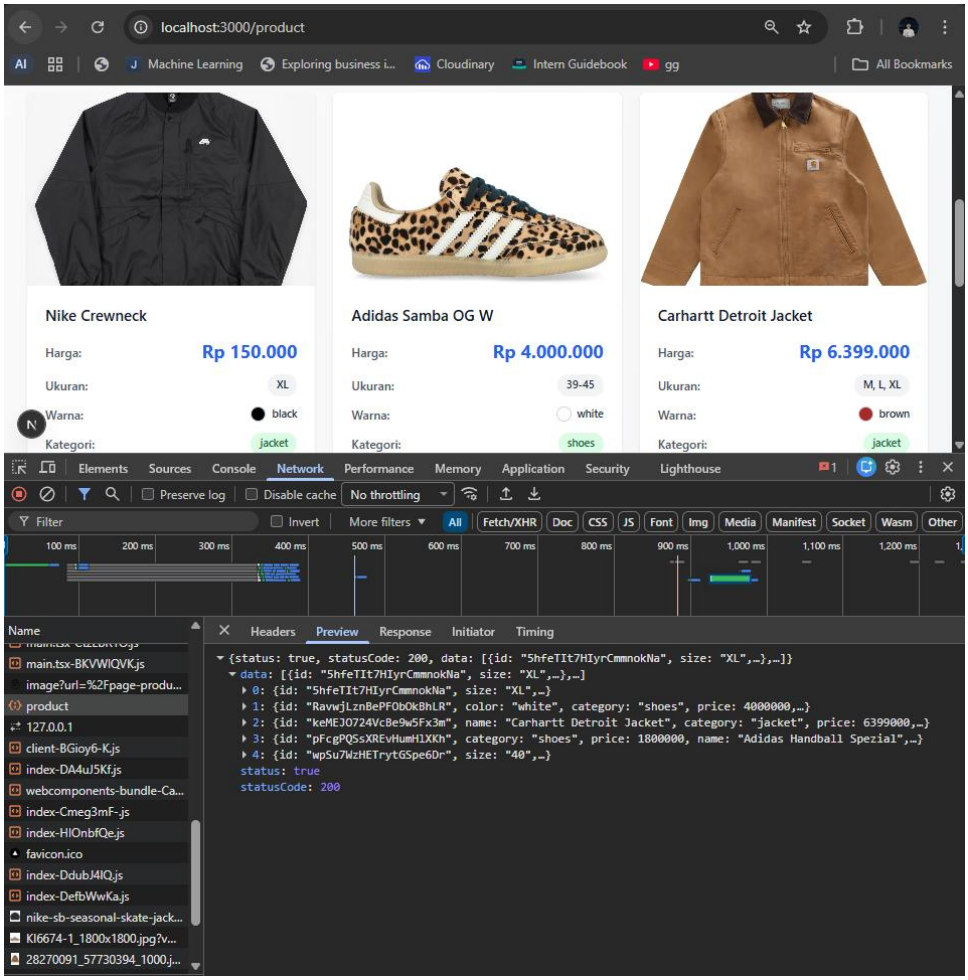
Akan tetap memunculkan semua product sebelum maupun sesudah di refresh

TUGAS PRAKTIKUM

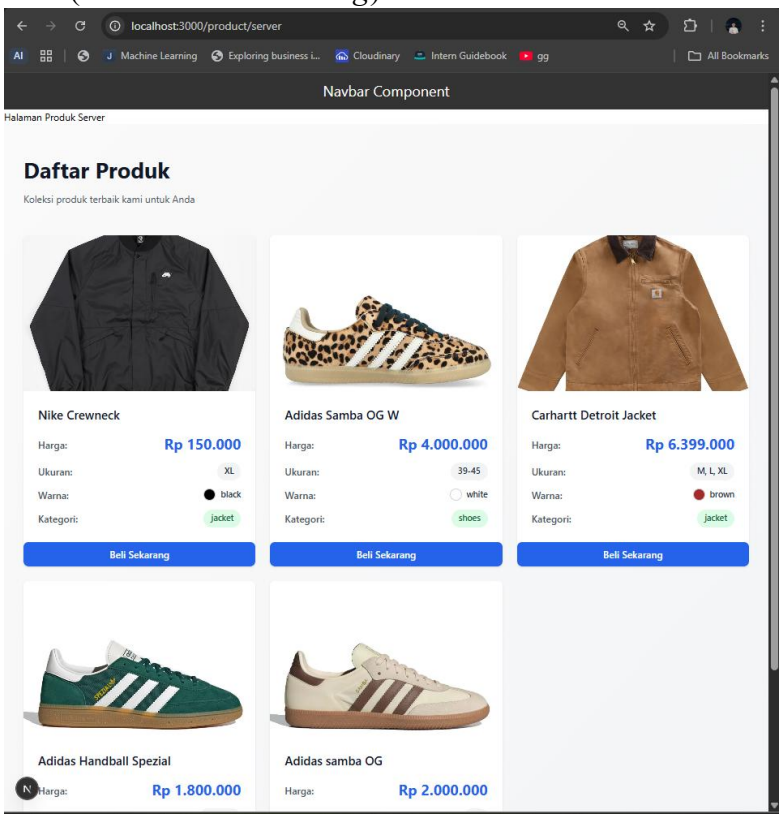
1. CSR (*Client-Side Rendering*)

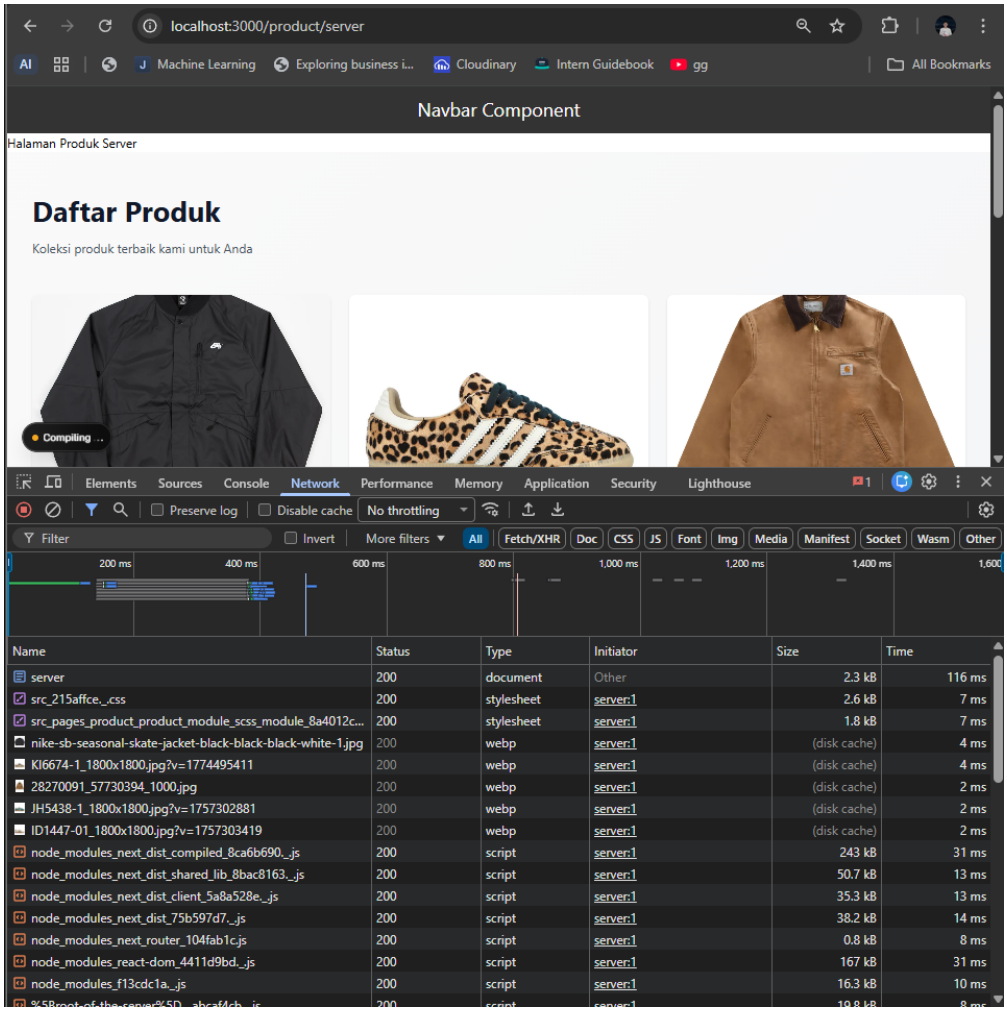






2. SSR (*Server-Side Rendering*)





3. Analisis:
   - Analisis Perilaku Rendering Visual (Uji 1: Pengamatan Skeleton)

     Berdasarkan dokumentasi praktikum pada Uji 1, ditemukan bahwa ketika halaman dengan arsitektur CSR (pada *route* /product) dimuat ulang (*refresh*), halaman tersebut akan memunculkan *skeleton* (kerangka pemuatan) untuk sementara waktu. Sebaliknya, ketika halaman dengan arsitektur SSR (pada *route* /product/server) dimuat ulang, *skeleton* tersebut sama sekali tidak muncul dan konten langsung tersedia di layar.

     munculnya *skeleton* pada CSR merupakan konsekuensi logis dari cara peramban (*browser*) merender halaman. Pada arsitektur CSR, peramban klien mengunduh bundel JavaScript dan HTML dasar terlebih dahulu. Pada titik ini, data produk belum tersedia. Oleh karena itu, antarmuka *skeleton* harus ditampilkan sebagai *placeholder* agar layar tidak terlihat kosong atau *freeze* (membeku) selagi aplikasi melakukan proses pengambilan data (*fetching*) secara asinkron (misalnya menggunakan useEffect). Hal ini memberikan indikator visual yang baik bagi pengguna bahwa aplikasi sedang bekerja.

     Di sisi lain, ketiadaan *skeleton* pada SSR membuktikan bahwa perakitan halaman telah diselesaikan seluruhnya di sisi server (*server-side*). Pada saat pengguna meminta halaman (melakukan *refresh*), server Next.js (Node.js) akan mengeksekusi logika pengambilan data (*fetch*) dan memasukkan data tersebut ke

dalam struktur komponen React *sebelum* merender HTML. *Browser* kemudian menerima dokumen HTML yang sudah "matang" dan lengkap, sehingga peramban hanya perlu menampilkannya secara instan tanpa perlu melewati fase pemuatan (*loading state*) di sisi klien.

- Analisis Mekanisme Data Fetching dan Keamanan (Uji 2: Network Tab / XHR)

  Pada Uji 2 yang dilakukan menggunakan alat pengembang (*DevTools*), pemantauan pada *Network Tab* dengan filter XHR (*XMLHttpRequest*) menunjukkan perbedaan yang signifikan. Saat halaman CSR di-refresh, *request* (permintaan) menuju API (seperti /api/product) terlihat dengan sangat jelas. Namun, ketika halaman SSR di-refresh, *request* API tersebut tidak terlihat sama sekali dan tidak ada produk dengan *type fetch* yang terekam oleh peramban.

  Perbedaan lalu lintas jaringan ini menyoroti batas pertanggungjawaban antara server dan klien. Pada CSR, *browser* klien secara langsung bertugas untuk memanggil API. Karena *request* tersebut diberangkatkan langsung dari peramban pengguna melintasi internet menuju peladen API, *Network Tab* dapat dengan mudah mencegat dan menampilkan detail alur lalu lintas data tersebut secara terbuka.

  Sebaliknya, pada implementasi SSR, tugas pemanggilan API ditransfer secara penuh ke server internal. Fungsi khusus seperti getServerSideProps dieksekusi di lingkungan *backend* Node.js. Server memanggil *endpoint* API, mengambil data, merakit HTML, lalu mengirimkan *output* akhirnya saja kepada klien. Karena *browser* klien tidak pernah melakukan komunikasi *request/response* secara asinkron untuk mendapatkan data tersebut, *Network Tab* klien tidak akan mencatat adanya pemanggilan *fetch*.

  Karakteristik SSR ini memberikan keunggulan masif dalam hal keamanan perangkat lunak (*security*). Dengan menyembunyikan lalu lintas pemanggilan data di sisi peladen, pengembang dapat dengan aman menyisipkan rahasia seperti API *Keys*, *Database Credentials*, maupun token autentikasi tanpa takut terekspos kepada publik melalui inspeksi *browser*.

- Analisis Struktur Kode dan Implikasi SEO (Uji 3: Response HTML / View Source)

  Uji 3 dilakukan dengan mengamati tulang punggung struktur *Response HTML* awal. Pada implementasi CSR, HTML awal yang dikirimkan terbukti kosong atau hanya berisi cangkang kerangka *skeleton* saja. Hal ini berbeda drastis dengan implementasi SSR, di mana HTML yang dikirimkan sejak awal sudah berisi data teks produk yang lengkap secara keseluruhan.

  Profil HTML awal yang kosong pada metode CSR terjadi karena pembangunan antarmuka (*Document Object Model*/DOM) sepenuhnya diserahkan kepada skrip JavaScript yang dieksekusi oleh klien pasca-unduh. Sayangnya, pendekatan ini menimbulkan masalah fatal terhadap SEO. Robot mesin pencari (*web crawlers* seperti Googlebot) yang bertugas mengindeks internet umumnya didesain untuk membaca struktur HTML mentah. Jika mereka merayapi halaman CSR, mereka hanya akan melihat "halaman kosong" atau *skeleton*, sehingga informasi mengenai produk (seperti nama, harga, dan gambar sepatu) tidak akan terindeks di mesin pencari.

Sebaliknya, SSR menyelesaikan seluruh komputasi konten di server, membuahkan dokumen HTML yang sarat akan teks dan data langsung. Ketika mesin pencari merayapi halaman SSR, mereka seketika dihidangkan dengan struktur data yang kaya dan siap diekstrak. Fakta analitik ini menjadikan SSR sebagai standar industri yang hampir diwajibkan bagi entitas situs web yang bergantung pada visibilitas publik, e-commerce, portal berita, atau blog, agar konten mereka dapat menempati peringkat teratas pada hasil penelusuran (SERP).

- Kesimpulan

  Berdasarkan serangkaian uji coba praktikum, terbukti secara empiris bahwa Client-Side Rendering dan Server-Side Rendering memiliki tujuan spesifik yang berbeda. Pendekatan CSR sangat ideal digunakan untuk membangun aplikasi web tertutup (dashboard admin, portal internal perusahaan) di mana SEO bukanlah prioritas. CSR memberikan transisi antarmuka yang sangat dinamis walau harus mengandalkan loading skeleton dan membebankan performa awal pada peranti pengguna.

  Di sisi lain, pendekatan SSR mutlak diperlukan apabila prioritas utama adalah performa SEO , kerahasiaan keamanan API , dan kecepatan persepsi (perceived performance) tanpa adanya waktu tunggu dari skeleton. Meskipun SSR menuntut daya komputasi server yang lebih berat karena setiap request harus dirender dari awal, keuntungannya terhadap pengindeksan data katalog sangatlah krusial bagi kesuksesan bisnis online.

STUDI ANALISIS

Jawab pertanyaan berikut:

1. Mengapa SSR lebih baik untuk SEO?

   SSR sangat optimal untuk SEO (*Search Engine Optimization*) karena server langsung mengirimkan dokumen HTML yang sudah dirender secara penuh dan berisi teks/data lengkap ke *browser*. Robot perayap mesin pencari (*web crawlers* seperti Googlebot) dirancang untuk membaca struktur HTML mentah. Dengan SSR, *crawler* dapat secara instan membaca, mengekstrak, dan mengindeks konten (seperti nama produk, deskripsi, dan harga). Sebaliknya, pada CSR, HTML awal yang dikirimkan umumnya kosong (hanya berupa kerangka/skeleton) karena kontennya baru dimuat oleh JavaScript setelah halaman tiba di *browser*, sehingga sering kali tidak terbaca oleh mesin pencari.

2. Kapan sebaiknya menggunakan SSR?
   - Sangat cocok untuk situs web yang menargetkan audiens publik dan bersaing di mesin pencari, seperti situs *E-commerce* (toko online), blog, portal berita, profil perusahaan, atau *landing page* promosi.
   - Ketika aplikasi harus menyajikan informasi secepat mungkin di layar pengguna pada muatan pertama (*initial load*) tanpa menampilkan status *loading*.
   - Karena proses *fetching* dilakukan di dalam server, kredensial keamanan seperti *API Keys* atau alamat *database* tidak akan terekspos di tab *Network browser* pengguna.

3. Apa kekurangan SSR dibanding CSR?
   - Pada arsitektur SSR, server harus bekerja ekstra untuk mengambil data dari API/database dan merakit HTML dari awal setiap kali ada request halaman dari pengguna. Hal ini membutuhkan daya komputasi server yang lebih besar.
   - *Browser* pengguna harus menunggu server menyelesaikan seluruh proses *rendering* HTML terlebih dahulu sebelum server memberikan respons. Hal ini bisa memakan waktu sepersekian detik lebih lama dibandingkan CSR yang langsung mengembalikan HTML kosong secara instan.
   - Untuk aplikasi web yang sangat interaktif dan tertutup (seperti *dashboard admin*, web portal internal, atau aplikasi *chatting* yang memerlukan autentikasi/login), CSR jauh lebih efisien karena perpindahan halamannya terasa lebih instan dan tidak membebani server secara terus-menerus.

4. Mengapa skeleton tidak muncul pada SSR?

   Skeleton tidak muncul pada SSR karena seluruh proses tunggu pengambilan data (*data fetching*) dan penjahitan antarmuka telah diselesaikan sepenuhnya di sisi server. Pada saat *browser* milik pengguna menerima respons jaringan, dokumen HTML tersebut sudah 100% "matang" dan berisi data nyata. Karena *browser* tidak lagi mengalami fase "menunggu data tiba" di sisi klien (seperti halnya menggunakan useEffect pada CSR), maka aplikasi tidak perlu lagi memicu status pemuatan (isLoading) untuk memunculkan antarmuka *skeleton*. Peramban hanya bertugas menampilkan gambar akhir seketika itu juga.